

## 10\_Phishing\_Detection

### Step 1: Import all libraries;

In this step, we import all the required Python libraries for machine learning, data processing, and evaluation. These include scikit-learn for model building, pandas and numpy for data handling, and matplotlib for visualization.

```
# ML for CBS practicals.
# Make a copy of this notebook and place it under your own
"MyDrive/Colab Notebooks" folder
# Please do not the run code here.

#import necessary libraries
from sklearn import tree
from sklearn.metrics import accuracy_score
import numpy as np
import pandas as pd
```

### Step 2: Load the dataset and read into variables

In this step, we mount Google Drive to Google Colab and load the dataset from the specified path. We also assign column names to the dataset as per the given schema, which allows for easier reference to the data fields.

```
#Mounting my google drive here in google colab using the following code
from google.colab import drive
drive.mount('/content/drive')
# Adding the path to train and test data
# Goto "MyDrive/Coab Notebooks" folder on your Google drive.
# Inside the folder, create folder structure
"Datasets/Datasets/Phishing_Datasets"
# Upload the downloaded csv data files to the folder.
path_to_data = "/content/drive/MyDrive/Colab
Notebooks/Datasets/Phishing_Datasets/dataset.csv"
#Adding the name of the columns shown by the hithub source

data_columns = ['having_ip', 'URL_len', 'Shortening_Service',
'having_At_symbol', 'double_slash_redirecting', 'prefix_suffix',
'having_sub_Domain', 'SSL_final_State', 'Domain_Registration_Len' ,
'Favicon', 'Port', 'HTTPS_Token', 'Request_URL', 'URL_of_Anchor',
'Links_in_Tags', 'SFH', 'Submitting_To_Emial', 'Abnormal_URL',
'Redirect', 'On_Mouseover', 'RightClick', 'Pop_Up_Window', 'Iframe',
```

```
'Age_of_Domain', 'DNS_Record', 'Web_Traffic', 'Page_Rank',
'Google_Index', 'Links_Pointing_to_Page',
'Statistical_Report', 'Labels']
```

```
#Reading the dataset by adding the data_columns
```

```
dataset = pd.read_csv(path_to_data, names = data_columns)
```

```
#Reading the dataset by adding the data_columns
```

```
dataset = pd.read_csv(path_to_data, names = data_columns)
```

```
#Reading the dataset by adding the data_columns
dataset = pd.read_csv(path_to_data, names = data_columns)
```



|       | having_ip | URL_len | Shortening_Service | having_At_symbol | double_slash_redirecting | prefix_suffix | having_sub_Domain | SSL_final_State | Domain |
|-------|-----------|---------|--------------------|------------------|--------------------------|---------------|-------------------|-----------------|--------|
| 0     | -1        | 1       | 1                  | 1                | -1                       | -1            | -1                | -1              |        |
| 1     | 1         | 1       | 1                  | 1                | 1                        | -1            | 0                 | 1               |        |
| 2     | 1         | 0       | 1                  | 1                | 1                        | -1            | -1                | -1              |        |
| 3     | 1         | 0       | 1                  | 1                | 1                        | -1            | -1                | -1              |        |
| 4     | 1         | 0       | -1                 | 1                | 1                        | -1            | 1                 | 1               |        |
| ...   | ...       | ...     | ...                | ...              | ...                      | ...           | ...               | ...             |        |
| 11050 | 1         | -1      | 1                  | -1               | 1                        | 1             | 1                 | 1               |        |
| 11051 | -1        | 1       | 1                  | -1               | -1                       | -1            | 1                 | -1              |        |
| 11052 | 1         | -1      | 1                  | 1                | 1                        | -1            | 1                 | -1              |        |
| 11053 | -1        | -1      | 1                  | 1                | 1                        | -1            | -1                | -1              |        |
| 11054 | -1        | -1      | 1                  | 1                | 1                        | -1            | -1                | -1              |        |

11055 rows x 31 columns

### Step 3: Explore the dataset

We explore the dataset to understand its structure and contents. This includes printing its shape (rows and columns), checking data types, generating descriptive statistics, and viewing the unique label values along with their counts

```
dataset.shape #printing the shape of the dataset using .shape function
```

**(11055, 31)**

```
dataset.info() #printing the attribute information infomation about teh
dataset
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11055 entries, 0 to 11054
Data columns (total 31 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   having_ip                             11055 non-null  int64
1   URL_len                               11055 non-null  int64
2   Shortening_Service                    11055 non-null  int64
3   having_At_symbol                       11055 non-null  int64
4   double_slash_redirecting              11055 non-null  int64
5   prefix_suffix                          11055 non-null  int64
6   having_sub_Domain                     11055 non-null  int64
7   SSL_final_State                       11055 non-null  int64
8   Domain_Registration_Len                11055 non-null  int64
9   Favicon                               11055 non-null  int64
10  Port                                  11055 non-null  int64
11  HTTPS_Token                           11055 non-null  int64
12  Request_URL                           11055 non-null  int64
13  URL_of_Anchor                          11055 non-null  int64
14  Links_in_Tags                          11055 non-null  int64
15  SFH                                    11055 non-null  int64
16  Submitting_To_Email                   11055 non-null  int64
17  Abnormal_URL                           11055 non-null  int64
18  Redirect                               11055 non-null  int64
19  On_Mouseover                           11055 non-null  int64
20  RightClick                             11055 non-null  int64
21  Pop_Up_Window                          11055 non-null  int64
22  Iframe                                 11055 non-null  int64
23  Age_of_Domain                          11055 non-null  int64
24  DNS_Record                             11055 non-null  int64
25  Web_Traffic                            11055 non-null  int64
26  Page_Rank                              11055 non-null  int64
27  Google_Index                           11055 non-null  int64
28  Links_Pointing_to_Page                 11055 non-null  int64
29  Statistical_Report                     11055 non-null  int64
30  Labels                                 11055 non-null  int64
dtypes: int64(31)
memory usage: 2.6 MB

```

```
dataset.describe() #generating basic statistics using .describe funtion
```

|       | having_ip    | URL_len      | Shortening_Service | having_At_symbol | double_slash_redirecting | prefix_suffix | having_sub_Domain | SSL_final_State | Domain       |
|-------|--------------|--------------|--------------------|------------------|--------------------------|---------------|-------------------|-----------------|--------------|
| count | 11055.000000 | 11055.000000 | 11055.000000       | 11055.000000     | 11055.000000             | 11055.000000  | 11055.000000      | 11055.000000    | 11055.000000 |
| mean  | 0.313795     | -0.633198    | 0.738761           | 0.700588         | 0.741474                 | -0.734962     | 0.063953          | 0.250927        | 0.250927     |
| std   | 0.949534     | 0.766095     | 0.673998           | 0.713598         | 0.671011                 | 0.678139      | 0.817518          | 0.911892        | 0.911892     |
| min   | -1.000000    | -1.000000    | -1.000000          | -1.000000        | -1.000000                | -1.000000     | -1.000000         | -1.000000       | -1.000000    |
| 25%   | -1.000000    | -1.000000    | 1.000000           | 1.000000         | 1.000000                 | -1.000000     | -1.000000         | -1.000000       | -1.000000    |
| 50%   | 1.000000     | -1.000000    | 1.000000           | 1.000000         | 1.000000                 | -1.000000     | 0.000000          | 1.000000        | 1.000000     |
| 75%   | 1.000000     | -1.000000    | 1.000000           | 1.000000         | 1.000000                 | -1.000000     | 1.000000          | 1.000000        | 1.000000     |
| max   | 1.000000     | 1.000000     | 1.000000           | 1.000000         | 1.000000                 | 1.000000      | 1.000000          | 1.000000        | 1.000000     |

8 rows x 31 columns

```
dataset['Labels'].unique() #extrcating the unique labels
```

```
array([-1,  1])
```

```
dataset['Labels'].value_counts() #value counts for all labels present in dataset
```

|        | count |
|--------|-------|
| Labels |       |
| 1      | 6157  |
| -1     | 4898  |

**dtype: int64**

#### Step 4: Preprocessing

Here, we preprocess the dataset by separating the features (independent variables) from the Labels (target variable). This prepares the dataset for training the machine learning model

```
#We Separate out the labels and data
```

```
features = dataset.drop('Labels', axis =1) #dropping the labels and
saving the features
```

```
Labels = dataset["Labels"] #saving the labels
```

```
features.shape #printing the shape of the save features
```

```
(11055, 30)
```

```
Labels.shape #printing the shape of save labels
```

```
(11055,)
```

```
features #printing the features
```

|       | having_ip | URL_len | Shortening_Service | having_At_symbol | double_slash_redirecting | prefix_suffix | having_sub_Domain | SSL_final_State | Domain_Regist |
|-------|-----------|---------|--------------------|------------------|--------------------------|---------------|-------------------|-----------------|---------------|
| 0     | -1        | 1       | 1                  | 1                | -1                       | -1            | -1                | -1              |               |
| 1     | 1         | 1       | 1                  | 1                | 1                        | -1            | 0                 | 1               |               |
| 2     | 1         | 0       | 1                  | 1                | 1                        | -1            | -1                | -1              |               |
| 3     | 1         | 0       | 1                  | 1                | 1                        | -1            | -1                | -1              |               |
| 4     | 1         | 0       | -1                 | 1                | 1                        | -1            | 1                 | 1               |               |
| ...   | ...       | ...     | ...                | ...              | ...                      | ...           | ...               | ...             |               |
| 11050 | 1         | -1      | 1                  | -1               | 1                        | 1             | 1                 | 1               |               |
| 11051 | -1        | 1       | 1                  | -1               | -1                       | -1            | 1                 | -1              |               |
| 11052 | 1         | -1      | 1                  | 1                | 1                        | -1            | 1                 | -1              |               |
| 11053 | -1        | -1      | 1                  | 1                | 1                        | -1            | -1                | -1              |               |
| 11054 | -1        | -1      | 1                  | 1                | 1                        | -1            | -1                | -1              |               |

11055 rows x 30 columns

```
features #printing the features
```

|       | having_ip | URL_len | Shortening_Service | having_At_symbol | double_slash_redirecting | prefix_suffix | having_sub_Domain | SSL_final_State | Doi |
|-------|-----------|---------|--------------------|------------------|--------------------------|---------------|-------------------|-----------------|-----|
| 0     | -1        | 1       | 1                  | 1                | -1                       | -1            | -1                | -1              |     |
| 1     | 1         | 1       | 1                  | 1                | 1                        | -1            | 0                 | 1               |     |
| 2     | 1         | 0       | 1                  | 1                | 1                        | -1            | -1                | -1              |     |
| 3     | 1         | 0       | 1                  | 1                | 1                        | -1            | -1                | -1              |     |
| 4     | 1         | 0       | -1                 | 1                | 1                        | -1            | 1                 | 1               |     |
| ...   | ...       | ...     | ...                | ...              | ...                      | ...           | ...               | ...             |     |
| 11050 | 1         | -1      | 1                  | -1               | 1                        | 1             | 1                 | 1               |     |
| 11051 | -1        | 1       | 1                  | -1               | -1                       | -1            | 1                 | -1              |     |
| 11052 | 1         | -1      | 1                  | 1                | 1                        | -1            | 1                 | -1              |     |
| 11053 | -1        | -1      | 1                  | 1                | 1                        | -1            | -1                | -1              |     |
| 11054 | -1        | -1      | 1                  | 1                | 1                        | -1            | -1                | -1              |     |

11055 rows x 30 columns

```
Labels #printing the labels
```

| Labels |     |
|--------|-----|
| 0      | -1  |
| 1      | -1  |
| 2      | -1  |
| 3      | -1  |
| 4      | 1   |
| ...    | ... |
| 11050  | 1   |
| 11051  | -1  |
| 11052  | -1  |
| 11053  | -1  |
| 11054  | -1  |

11055 rows x 1 columns

dtype: int64

## Step 5: Preprocessing

We split the dataset into training and testing subsets. The first 2000 rows are used for training (X\_train, Y\_train) and the remaining rows are used for testing (X\_test, Y\_test). This allows for performance evaluation on unseen data.

```
# Separate the training and testing data
X_train = features[:2000] #storing the first 2000 samples/instances in
X_train
Y_train = Labels[:2000] #Stroing their respective training labels in
Y_train
X_test = features[2000:] #stroing the remaining samples/instances in
X_test
```

```
Y_test = Labels[2000:] #Storing their respective testing labels in Y_test
```

```
X_train.shape #printing x_train using .shape  
(2000, 30)
```

```
X_train #printing x_train
```

```
having_ip  URL_len  Shortening_Service  having_At_symbol  double_slash_redirecting  prefix_suffix  having_sub_Domain  SSL_final_State  Domain
0         -1       1                   1                   1                   -1                   -1                   -1                   -1
1         1       1                   1                   1                   1                   -1                   0                   1
2         1       0                   1                   1                   1                   -1                   -1                   -1
3         1       0                   1                   1                   1                   -1                   -1                   -1
4         1       0                   -1                  1                   1                   -1                   1                   1
...      ...      ...                  ...                  ...                  ...                  ...                  ...                  ...
1995      1      -1                   1                   1                   1                   -1                   -1                   -1
1996      1      -1                   1                   1                   1                   -1                   1                   1
1997      1      -1                   1                   1                   1                   -1                   -1                   -1
1998      1      -1                   1                   1                   1                   -1                   1                   1
1999      1      -1                   1                   1                   1                   -1                   0                   0
```

2000 rows x 30 columns

```
Y_train.shape #printing the shape y_train using .shape
```

```
(2000,)
```

```
Y_train #print the y_train
```

```
Labels
0      -1
1      -1
2      -1
3      -1
4       1
...     ...
1995    -1
1996    -1
1997    -1
1998     1
1999    -1
```

2000 rows x 1 columns

dtype: int64

```
X_test.shape #printing X_test shape using .shape  
(9055, 30)
```

```
X_test #printing X_test
```

|       | having_ip | URL_len | Shortening_Service | having_At_symbol | double_slash_redirecting | prefix_suffix | having_sub_Domain | S   |
|-------|-----------|---------|--------------------|------------------|--------------------------|---------------|-------------------|-----|
| 2000  | 1         | -1      | 1                  | 1                | 1                        | -1            | -1                | -1  |
| 2001  | 1         | -1      | 1                  | 1                | 1                        | -1            | 0                 | 0   |
| 2002  | 1         | 1       | 1                  | 1                | 1                        | -1            | 0                 | 0   |
| 2003  | 1         | -1      | 1                  | 1                | 1                        | -1            | 0                 | 0   |
| 2004  | 1         | 1       | 1                  | 1                | 1                        | -1            | -1                | -1  |
| ...   | ...       | ...     | ...                | ...              | ...                      | ...           | ...               | ... |
| 11050 | 1         | -1      | 1                  | -1               | 1                        | 1             | 1                 | 1   |
| 11051 | -1        | 1       | 1                  | -1               | -1                       | -1            | 1                 | 1   |
| 11052 | 1         | -1      | 1                  | 1                | 1                        | -1            | 1                 | 1   |
| 11053 | -1        | -1      | 1                  | 1                | 1                        | -1            | -1                | -1  |
| 11054 | -1        | -1      | 1                  | 1                | 1                        | -1            | -1                | -1  |

```
Y_test.shape #Printing Y_test.shape using .shape
(9055,)
```

```
Y_test #printing Y_test
```

| Labels |     |
|--------|-----|
| 2000   | -1  |
| 2001   | -1  |
| 2002   | 1   |
| 2003   | -1  |
| 2004   | 1   |
| ...    | ... |
| 11050  | 1   |
| 11051  | -1  |
| 11052  | -1  |
| 11053  | -1  |
| 11054  | -1  |

9055 rows x 1 columns

dtype: int64

## Step 6: Train a classifier model: Decision

In this step, we initialize and train a Decision Tree Classifier using the training data. The model learns patterns that differentiate phishing from non-phishing websites.

```
# Create a decision tree classifier model using scikit-learn
Attack_Detection_Model = tree.DecisionTreeClassifier()
print("Decision tree classifier created.")
```

➤ Decision tree classifier created.

```
# Train the decision tree classifier
Attack_Detection_Model.fit(X_train, Y_train)
print ("Model training completed.")
```

➤ Model training completed.

Step 7: Evaluate the classification model

We evaluate the trained model by performing cross-validation on the training data, making predictions on the test set, and calculating metrics such as accuracy, confusion matrix, precision, recall, and F1-score. We also visualize the trained decision tree using Graphviz.

```
from sklearn.model_selection import cross_validate #import additional
library for cross validation
scores = cross_validate(Attack_Detection_Model, X_train, Y_train,
cv=10, return_train_score=False)
print(scores) #print the scores
{'fit_time': array([0.01366448, 0.00832677, 0.01577258, 0.03842115, 0.01899147,
0.02305388, 0.0232482 , 0.01620102, 0.00860715, 0.00862193]), 'score_time': array([0.00359249, 0.00475931, 0.00319552, 0.018502 , 0.00885344,
0.01265979, 0.01043177, 0.00850439, 0.00346518, 0.00370526]), 'test_score': array([0.965, 0.94 , 0.965, 0.975, 0.925, 0.94 , 0.955, 0.95 , 0.975,
0.955])}
```

```
# Use the trained classifier to make predictions on the test data
predictions = Attack_Detection_Model.predict(X_test)
print(predictions)
print("Predictions on testing data computed.")
```

➤ [-1 -1 1 ... 1 -1 -1]  
Predictions on testing data computed.

```
# Print the accuracy (percentage of phishing websites correctly
predicted)
accuracy = 100.0 * accuracy_score(Y_test, predictions)
print("The accuracy of your decision tree on testing data is: " +
str(accuracy))
➤ The accuracy of your decision tree on testing data is: 90.6791827719492
```

```
from sklearn.metrics import confusion_matrix, classification_report #
print(confusion_matrix(Y_test, predictions))
```



